



Vacation Rental Marketplace — Production Architecture Document

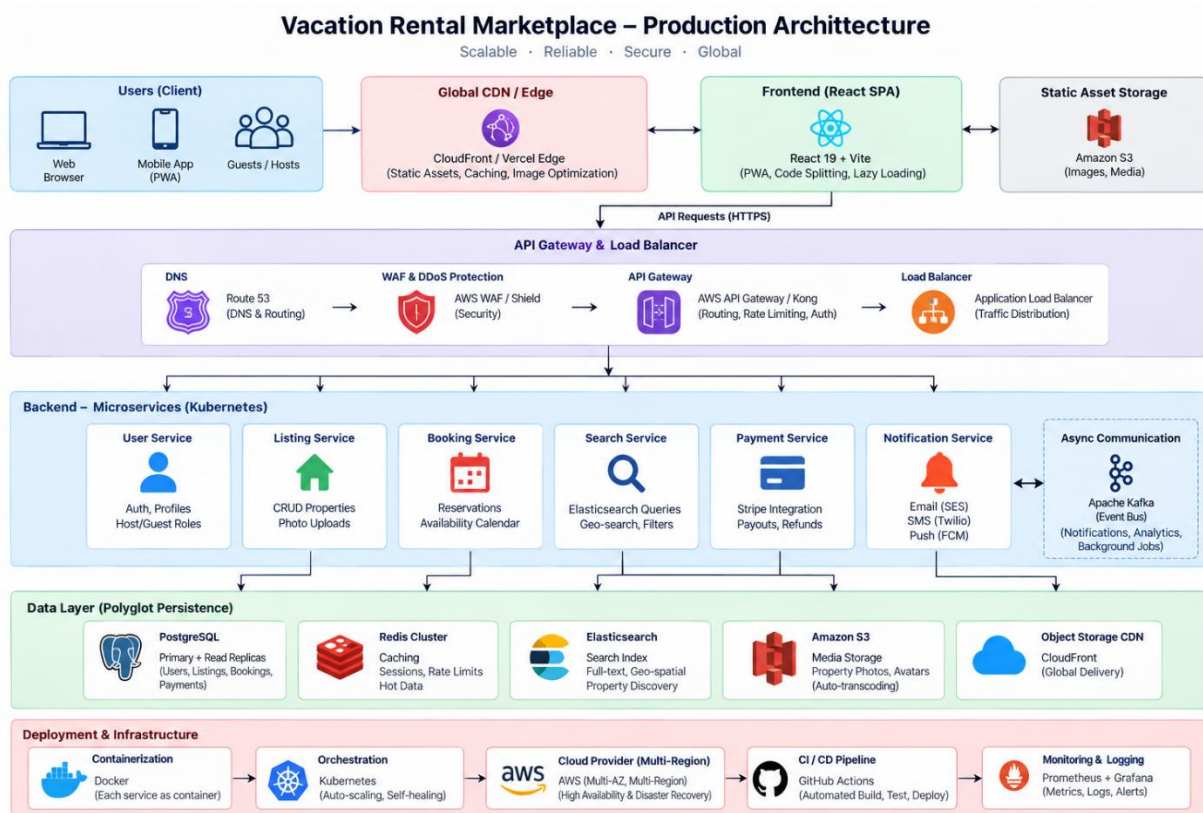
Project: Airbnb Clone (React + Vite)

Author: Airbnb Antigravity Project

Date: September 2026

Stack: React 19 · Vite · JavaScript · Vanilla CSS

Figure 1: High-level 5-layer production architecture for a vacation-rental marketplace at Airbnb scale.



1. Overview

This document describes the production-scale architecture for a vacation-rental marketplace — a platform enabling guests to discover, book, and review properties, while hosts can list and manage their rentals.

The architecture is designed around five core scaling pillars:

Pillar	Strategy
Frontend	React SPA with CDN-edge delivery and PWA support
Backend	Domain-driven microservices behind an API Gateway
Storage	Polyglot persistence (PostgreSQL + Redis + S3)
Search	Elasticsearch with geo-spatial indexing
Deployment	Kubernetes on multi-region cloud with CI/CD

2. Layer-by-Layer Breakdown

Layer 1 — Client Tier

The frontend is a React 19 Single-Page Application built and bundled with Vite, delivered globally via a CDN.

Component	Technology	Purpose
React SPA	React 19 + Vite	Core UI — listing detail, booking flow, host profiles
PWA Support	Service Workers + Manifest	Offline mode, installable shell, background sync
CDN Static Assets	CloudFront / Vercel Edge	JS/CSS/image delivery with <50ms TTFB globally
Global CDN	AWS CloudFront or Vercel	Routes all traffic; geo-aware edge caching

Scaling Strategy

- The entire React build is static — zero server-side rendering cost for page loads.
- Images are served via CDN with automatic WebP/AVIF transcoding.
- The bundle is split by route (React.lazy + Suspense) so the initial JS payload stays under 200 KB.
- A Service Worker pre-caches the app shell, enabling near-instant repeat visits.

Layer 2 — API Gateway & Load Balancing

All client API calls are routed through a unified entry point before reaching any microservice.

Component	Technology	Purpose
API Gateway	AWS API Gateway / Kong	Single entry point, routing, versioning (/v1/, /v2/)
Load Balancer	AWS ALB / Nginx	Distributes traffic across healthy service instances
Rate Limiter	Redis-backed Token Bucket	1,000 req/min per IP for guests; 5,000 for authenticated users
Auth Service	JWT + OAuth 2.0 (Google/Apple SSO)	Issues short-lived access tokens; refresh-token rotation

Scaling Strategy

- The API Gateway is serverless (AWS API Gateway) — scales to millions of requests/sec with no ops overhead.
- Rate limiting state is stored in a shared Redis cluster so all pods enforce the same limits.
- JWT validation happens at the gateway — microservices receive pre-authenticated requests, eliminating redundant auth logic.

Layer 3 — Microservices

The backend is split into six bounded domains, each independently deployable:

Service	Responsibilities
Listing Service	CRUD for properties, photo uploads, amenity tagging
Booking Service	Reservation state machine, availability calendar
User/Auth Service	Registration, login, profile, host/guest roles
Search Service	Elasticsearch queries, geo-search, filters, ranking
Payments Service	Stripe integration, payout scheduling, refunds
Notification Service	Email (SES), SMS (Twilio), push (FCM) via event bus

Scaling Strategy

- Each service is packaged as a Docker image and runs on Kubernetes pods.
- Services communicate asynchronously via Kafka for non-critical paths (notifications, analytics) and synchronously via REST/gRPC for real-time needs (availability check during booking).
- The Booking Service uses an optimistic locking pattern with PostgreSQL advisory locks to prevent double-bookings without a distributed lock manager.
- The Search Service is horizontally scaled independently — Elasticsearch nodes autoscale based on query throughput.

Layer 4 — Data Layer (Polyglot Persistence)

Store	Technology	Used For
Primary DB	PostgreSQL 16 (Primary + 2 Read Replicas)	Users, listings, bookings, payments — transactional data
Cache	Redis Cluster (6 shards)	Sessions, rate-limit counters, hot listing data, availability calendars
Search Index	Elasticsearch 8	Full-text search, geo-spatial property discovery, faceted filtering
Media Storage	AWS S3 + CloudFront CDN	Property photos, host avatars — with auto-transcoding pipeline

Store	Technology	Used For
Message Queue	Apache Kafka	Event streaming: booking created → notification → analytics

Scaling Strategy

PostgreSQL:

- Write traffic → Primary node only; Read traffic → 2 read-replicas via PgBouncer connection pool
- Point-in-time recovery via continuous WAL archiving to S3
- Long-term archival → partitioned tables for analytics

Redis:

- 6-node cluster with consistent hashing
- TTL-based eviction for cached listing pages (5-minute TTL)
- Availability calendars cached per-listing, invalidated on booking

Elasticsearch:

- Separate index per entity type (listings, reviews)
- Geo-point fields enable radius-based search
- Updated via Kafka consumer (Listing Service → Kafka → ES Indexer)

AWS S3:

- Lifecycle rules: move images > 90 days to S3 Glacier Instant Retrieval
- Lambda trigger on upload → image resize pipeline (WebP/AVIF)
- CloudFront signed URLs for private host documents

Layer 5 — Infrastructure & Deployment

Component	Technology	Purpose
Container Orchestration	Kubernetes (EKS)	Pod autoscaling, rolling deployments, self-healing
CI/CD Pipeline	GitHub Actions	Test → Build → Push ECR → Deploy to K8s on every merge to main
Monitoring	Prometheus + Grafana	Latency histograms, error rates, pod CPU/memory dashboards
Logging	ELK Stack	Centralized structured logs across all services
Tracing	OpenTelemetry + Jaeger	Distributed request tracing across microservices
Multi-Region	AWS us-east-1 (primary) + eu-west-1 (secondary)	Active-passive failover with Route 53 health checks

Component	Technology	Purpose
Secret Management	AWS Secrets Manager	DB credentials, Stripe keys — never in environment variables

Scaling Strategy

- **Horizontal Pod Autoscaler (HPA):** Scales service replicas based on CPU utilization (target 60%).
- **Cluster Autoscaler:** Adds/removes EC2 nodes based on pending pod pressure.
- **Blue-Green Deployments:** Zero-downtime releases by routing traffic between two identical deployment slots.
- **Multi-Region Failover:** Route 53 latency-based routing; if us-east-1 health check fails, traffic switches to eu-west-1 within 60 seconds.

3. Current Implementation (This Repository)

This repository is a high-fidelity frontend prototype demonstrating the Client Tier of the full production system.

Component Architecture

App.jsx (Root)	
Header.jsx	
StickyNav.jsx	(IntersectionObserver-based active section tracking)
features/	
listing/	
HeroGrid	(5-image masonry hero)
PropertyDetails	(Full detail left column)
TitleRow	(Property title & badges)
Location	(OpenStreetMap embed)
ThingsToKnow	(Policies 3-column grid)
PhotoTour	(Full-screen photo gallery overlay)
Lightbox	(Single-image viewer with keyboard nav)
AmenitiesModal	(Full amenity list modal)
booking/	
BookingWidget	(Sticky pricing card + date/guest form)
reviews/	
Reviews	(Rating bars + keyword chips + review cards)
host/	
HostProfile	(Host stats + co-host details)
explore/	
SimilarListings	(Horizontal carousel with pagination)
data/listing.json	(Static data store – mocks API response)
imageProxy.js	(CDN URL resolver)

State Management

State	Location	Mechanism
Modal Visibility (PhotoTour, Lightbox, Amenities)	App.jsx	useState
Current Lightbox Image Index	App.jsx	useState

State	Location	Mechanism
Booking Dates & Guests	BookingWidget.jsx	Local useState
Active Nav Section	StickyNav.jsx	IntersectionObserver

4. Scaling Strategy Summary

Frontend Scaling

Challenge	Solution
High traffic spikes	CDN caching + edge workers serve 99% of traffic without hitting origin
Large media files	Auto-transcoded WebP via S3 Lambda pipeline; responsive <picture> tags
SEO for crawlers	Optional Next.js SSR layer or React static pre-rendering at build time
Mobile performance	PWA with offline shell; code-split bundles; lazy-loaded below-fold sections

Backend Scaling

Challenge	Solution
Booking contention (same dates)	Optimistic locking + PostgreSQL advisory locks
Search latency	Elasticsearch with pre-computed geo-indexes; response cached in Redis
Payment reliability	Stripe idempotency keys; Kafka-backed retry queue for failed webhooks
Notification throughput	Kafka consumer group; email batched via SES

Storage Scaling

Challenge	Solution
DB read throughput	2 PostgreSQL read replicas + Redis L2 cache
Image storage growth	S3 lifecycle to Glacier; CloudFront caching with long TTLs
Search relevance	Elasticsearch ML ranking models; user click-through signal feedback

Deployment Scaling

Challenge	Solution
Zero-downtime releases	Kubernetes rolling update + readiness probes
Regional outage	Active-passive multi-region with Route 53 failover
Cost efficiency	Kubernetes HPA scales pods down during low traffic

5. Technology Choices Rationale

Decision	Why
React + Vite over Next.js	Faster dev iteration for SPA prototype; CDN serves static build
PostgreSQL over MySQL	Native JSON columns, PostGIS for geo, better MVCC concurrency
Elasticsearch over Algolia	Self-hosted = zero per-query cost at scale; full geo-spatial support
Kafka over SQS	Replayable event log; multiple consumers (analytics + notifications) from same event
Redis over Memcached	Richer data types (sorted sets for availability windows, pub/sub for real-time)
Kubernetes over ECS	Vendor-agnostic; portability between AWS/GCP if needed
Stripe for payments	PCI-DSS compliance handled by Stripe; Connect for host payouts

6. Security Considerations

- **Authentication:** JWT with 15-minute expiry + refresh token rotation in HTTP-only cookies.
- **Authorization:** RBAC roles (guest, host, admin) enforced at the API Gateway and re-validated in each service.
- **Data Encryption:** All data encrypted at rest (AWS KMS) and in transit (TLS 1.3).
- **PII Handling:** Guest contact info masked until booking confirmed; GDPR right-to-erasure automated.
- **DDoS Protection:** AWS Shield Standard + WAF rules on the API Gateway.
- **Secrets:** No secrets in code or env files — all pulled from AWS Secrets Manager at pod startup.

7. Sequence of AI-Assisted Development Prompts

The following is the complete sequence of prompts used for AI-assisted development:

1. Claude - The file attached is task given to me, go through entire file, also given the file containing all the frontend images of Airbnb. Give a good high quality "MASTER PROMPT" that i can give to "v0.dev". Use Reactjs as Frontend and Node.js as backend. The Things i will provide to v0.dev - the generated Master prompt and the file which contain all the images of frontend of airbnb. Additionally also give things said in file like Architecture Diagram ie - Submit a high-level architecture diagram for a production-scale vacation-rental marketplace (think Airbnb) alongside your app. The diagram should illustrate your scaling strategy for frontend, backend, storage, search, and deployment. Tools like lucid.app or excalidraw are recommended.

2. v0.dev — Master prompt given to v0.dev, which was given by claude:

You are building a PIXEL-PERFECT clone of a real Airbnb listing page for a take-home assignment. Visual and behavioral fidelity to the attached reference screenshots is the #1 priority — this is being graded on how precisely it matches, not on creativity.

Reference listing: "Romantic Jacuzzi 1BHK Candolim | Mirashya UG10" – an entire serviced apartment in Candolim, Goa, India. I'm attaching full-page screenshots (top to bottom) of the exact reference site – treat them as the single source of truth for layout, spacing, typography, colors, icons and copy. Match them exactly, don't improvise or "improve" the design.

TECH STACK

- Framework: Next.js 14+ (App Router), TypeScript.
- Styling: Tailwind CSS, matching Airbnb's actual design tokens (see below).
- State: React state / URL search params for gallery, dates, guest picker – no backend calls needed for these, but structure the code so a Node.js/Express API could later be swapped in (see API layer notes at the end).
- Icons: lucide-react or a hand-rolled icon set matching Airbnb's line-icon style (thin 1.5px stroke, rounded caps) – do NOT use filled/solid icon styles.
- Fonts: Circular / Airbnb Cereal is proprietary – substitute the closest free equivalent (e.g. "Inter" or "Sofia Pro" fallback) but preserve the exact type scale, weight (600/700 for headings, 400/500 for body), and letter spacing shown in the screenshots.
- Deployable to Vercel with zero extra config.

BUILD THREE VIEWS (all desktop-only, no mobile breakpoint needed):

i) LISTING PAGE (default route "/")

Build top to bottom, exactly matching the reference:

- a. Sticky top navbar: Airbnb logo + wordmark (left), centered pill-shaped search bar ("Anywhere | Anytime | Add guests" with a small red circular search icon button), right side has "Become a host" text link, a globe icon button, and a rounded menu/hamburger icon button. On scroll past the photo grid, this navbar collapses into a slim sticky bar showing: section tabs (Photos / Amenities / Reviews / Location, active tab underlined), and on the right a compact price ("₹28,499 for 5 nights" + star rating + review count) next to a "Reserve" pill button.
- b. Title row: large bold H1 listing title ("Romantic Jacuzzi 1BHK Candolim | Mirashya UG10"), with "Share" (upload icon) and "Save" (heart outline icon) text-links aligned right, underlined on hover.
- c. Photo grid: one large hero photo on the left (rounded corners, ~50% width), and a 2x2 grid of four smaller photos on the right, all with matching rounded corners and thin gaps. The bottom-right tile has a white pill button overlay "田 Show all photos" centered on it. Hovering any tile should subtly darken/brighten with a smooth overlay transition (150–200ms ease).
- d. Below the grid, two-column layout (content ~60% left, sticky booking card ~40% right):
LEFT COLUMN:
 - "Entire serviced apartment in Candolim, India" (H2) + guest/bedroom/bed/bathroom summary line.
 - Divider.
 - "Guest favourite" info row: a small laurel/badge icon, "One of the most loved homes on Airbnb, according to guests", and to the right a star rating "4.95" with 5 filled stars, and "19 Reviews" as a separate stat.
 - Divider.
 - Host row: circular host avatar, "Hosted by Mirashya Homes", "2 years hosting".
 - Divider.
 - 3 highlight rows, each with a thin line icon + bold title + gray subtitle (e.g. "Outdoor entertainment / The pool and alfresco dining are great for summer trips", "Designed for staying cool", "Self check-in").
 - Divider.
 - A gray info banner: "Some info has been automatically translated. Show original" (underlined link).

- Description paragraph with emoji (🌴🌟🛀📺🐾🍹🌸❤️) exactly as shown, truncated with a "Show less / Show more" underlined toggle + chevron icon that expands/collapses the paragraph.
 - Divider, "Where you'll sleep" H2 + two photo cards side by side (Bedroom / 1 double bed, Living room / 1 sofa), rounded corners.
 - Divider, "What this place offers" H2 + two-column checklist of amenities with line icons (Kitchen, WiFi, Dedicated workspace, Free parking on premises, Pool, Hot tub, Pets allowed, Exterior security cameras) – two items shown with strikethrough text + a "no" icon (Carbon monoxide alarm, Smoke alarm, both "not reported"). Below: outlined pill button "Show all 50 amenities" that opens a modal/drawer listing all amenities grouped by category.
 - Divider, "5 nights in Candolim" H2 + date range subtitle, then a two-month calendar (October 2026 / November 2026) with day-of-week headers, selected range 18–23 Oct shown as filled black circular start/end dates with a connecting gray range background, past/unavailable dates grayed out and struck through, left/right chevron arrows to page months, a "Clear dates" underlined link, and a small calendar/list-view toggle icon.
 - Large centered laurel "4.95" rating mark, then Reviews section: "Guest favourite" H2, description line, "How reviews work" underlined link, a row of 7 rating categories (Overall rating as a horizontal bar chart 1–5, then Cleanliness/Accuracy/Check-in/Communication/Location/Value each as a number with a small icon), a horizontally scrollable row of tag pills with counts (Comfort 6, Accuracy 5, Hot tub 5, Condition 4, Hospitality 8, Cleanliness 4, Amenities 2, ...), then a 2-column grid of review cards (circular avatar or colored initial-avatar, name, "X on Airbnb", star rating, "time ago", review text truncated with "Show more" link). End with an outlined pill "Show all 19 reviews" button that opens a modal with the full review list.
 - Divider, "Where you'll be" H2, "Candolim, Goa, India" subtitle, a stylized static map placeholder (soft green/blue shapes as land/water, faint grid lines, a black circular pin with a house icon centered, +/- zoom control pills top right, a circular search icon top left), "Exact location will be provided after booking" caption, then "Neighbourhood highlights" heading + text + "Show more" link.
 - Divider, "Meet your host" H2: a rounded white card (host avatar, verified checkmark badge, "Mirashya Homes", "Host", stats column showing "1,463 Reviews / 4.68★ Rating / 2 Years hosting"), below it two bullet facts ("Born in the 80s", "Where I went to school: NICMAR GOA"), then a "Co-Hosts" list of avatars+names in a grid, "Host details" (Response rate 100%, Responds within an hour), a "Message host" button, and a small shield-icon safety note about paying only through Airbnb.
 - Divider, "Things to know" H2, 3-column layout (Cancellation policy / House rules / Safety & property), each with an icon, bolded sub-items, and "Learn more" underlined links.
 - Divider, "More stays nearby" H2 with pagination controls (page indicator "1 / 2", left/right circular chevron arrow buttons), a horizontally-laid-out row of 5 property cards (rounded photo, title, price, star rating), clicking the arrows swaps to a second set of 5 cards.
- RIGHT COLUMN (sticky booking card, stays pinned while scrolling the left column, only within the photo-grid-to-reviews scroll range):
- Small white bordered card: green tag icon, "Get 10% off your next stay." + "Terms apply" underlined link, "Claim" button on the right.
 - Larger white bordered card with shadow: price "₹28,499 for 5 nights", a 2x1 grid of "CHECK-IN 10/18/2026" / "CHECKOUT 10/23/2026" boxes with hover border highlight (clicking opens the calendar), a "GUESTS 2 guests" row with a chevron-down icon (opens a guest-counter dropdown), a gray "Free cancellation before 17 October" banner, a full-width pink-to-red gradient "Reserve" button (rounded, bold white text), and a centered gray caption "You won't be charged yet".

- Below the card, a small "🔴 Report this listing" underlined link.

ii) PHOTO TOUR (full-screen overlay, opened by clicking the hero image or "Show all photos"): a full-viewport white overlay with a close (X) button top-left, "Share" / "Save" top-right, left sidebar/rail listing photo category thumbnails (Living room 1, Living room 2, Full kitchen, Bedroom, Full bathroom, Pool, Exterior, Yard, etc.), and a main scrolling column of large photos each captioned with a room name + short description (e.g. "Living room 1 / Sofa, Air conditioning, Ceiling fan, TV"). Clicking any photo opens the Lightbox.

iii) LIGHTBOX (single-photo viewer opened from any photo tour image): full-screen dark/black backdrop, the photo centered and scaled to fit, thin circular prev/next chevron-arrow buttons on either side (hover = slight scale/opacity change), a close (X) button top-right, a photo counter or caption bottom-left, and smooth crossfade/slide transition (200-300ms) between photos. Must support LEFT/RIGHT arrow keys for navigation and ESC to close, with correct focus management (focus returns to the trigger element on close) and ARIA labels on all icon-only buttons for accessibility.

INTERACTION & MOTION DETAILS (grade heavily on this)

- All hover states use Airbnb's real timing: ~150-200ms ease-in-out transitions on color/background/border/shadow/scale.
- Buttons: subtle scale(0.97) or darken on :active.
- Sticky navbar transforms/shrinks smoothly on scroll (no jerk), and section tabs (Photos/Amenities/Reviews/Location) scroll-spy to highlight the active section and smooth-scroll into view on click.
- Calendar, guest-picker, and "show all photos/amenities/reviews" all open with a smooth fade+scale-in modal/drawer transition, and trap focus while open.
- Full keyboard navigation and visible focus rings on every interactive element; correct semantic HTML (headings hierarchy, button vs link usage, alt text on all images) and ARIA roles for modals/dialogs.

DATA

- Hardcode all listing content (title, photos as placeholder/stock URLs, price, dates, host info, amenities, reviews, nearby listings) as a typed mock dataset in a single lib/data.ts file, structured as if it came from a REST API (e.g. GET /api/listings/:id), so it can be swapped for a real Node.js/Express backend later without touching UI components.

DELIVERABLE

Return clean, componentized Next.js code (separate components per section: Navbar, PhotoGrid, PhotoTour, Lightbox, ListingHeader, HostInfo, Amenities, Calendar, Reviews, LocationMap, HostCard, ThingsToKnow, NearbyListings, BookingCard), with Tailwind for styling, TypeScript types for all data, and no console errors or warnings.

3. Antigravity - Then performed a lot of modifications by taking screenshots and giving to antigravity one by one to match the UI with the original Airbnb page. In each and every section, I asked Antigravity to modify the UI to match the original Airbnb page.

Built with React 19, Vite, and Google Antigravity AI assistance.